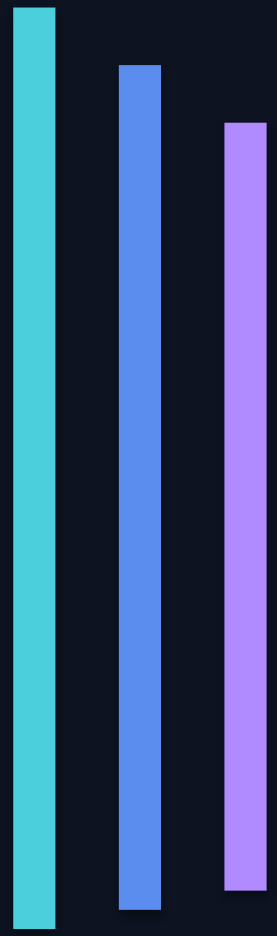
SISTEMAS DISTRIBUÍDOS • PROJETO

Jogo da velha em arquitetura distribuída

RMI para pareamento · TCP para jogadas · Multicast UDP para displays



Implementação em Java

Servidor de match • Jogador • Visualizador

Samuel Rocha Castanheira, Matheus Luiz da Cruz Morata Marinho | Repositório: github.com/SamuelCastanheira/rmi

Separar responsabilidades e canais de comunicação

O jogo é simples; o desafio é coordenar processos em máquinas diferentes.

Servidor de match

Recebe pedidos, encontra dois jogadores compatíveis e define quem começa.

Jogador

Consulta o servidor, joga contra o oponente e publica cada jogada.

Visualizador

Recebe atualizações por multicast e mostra o tabuleiro em tempo real.

Uma tecnologia por necessidade

RMI

Chamada remota para solicitar o match.

TCP

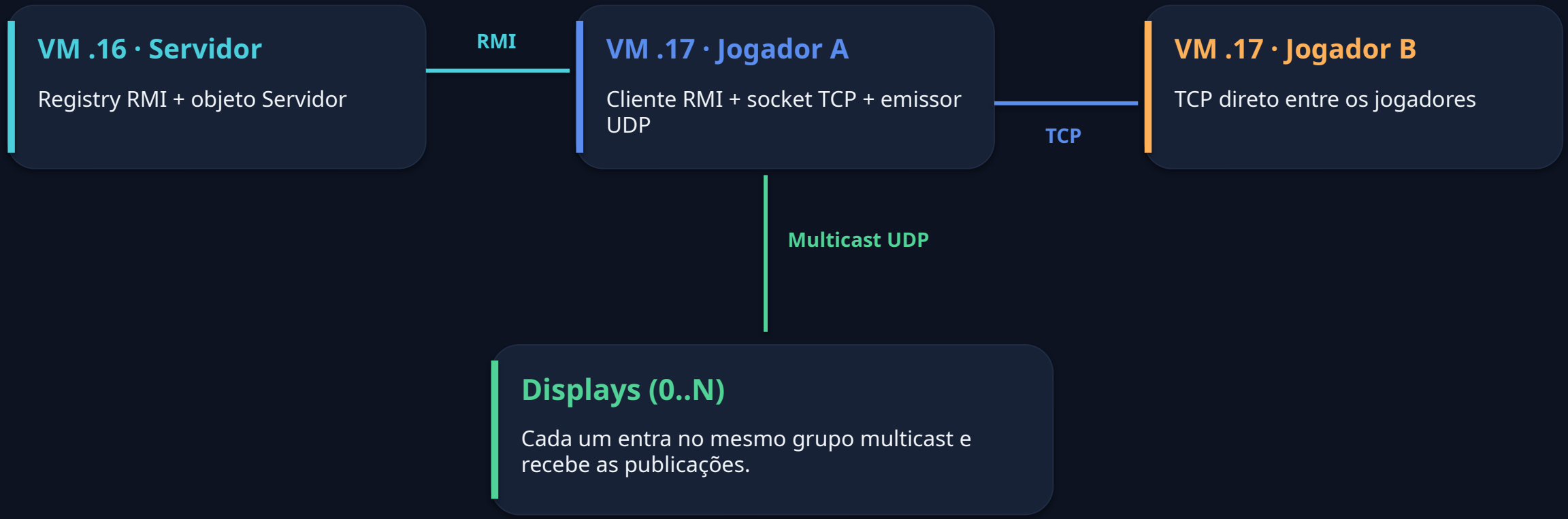
Canal confiável e ordenado entre os dois jogadores.

Multicast UDP

Uma publicação pode chegar a vários displays.

Três fluxos, três responsabilidades

Topologia utilizada na execução: servidor na VM .16 e jogador(es) em outras VMs.



RMI não transporta as jogadas: ele apenas faz o pareamento inicial.

Pareamento: o método remoto bloqueia até encontrar par

O contrato remoto é pequeno: getOponent recebe JSON e devolve JSON.

TRECHO DO CÓDIGO

```
public interface IServidor extends Remote {
    String getOponent(String jsonIn)
        throws RemoteException;
}
```

TRECHO DO CÓDIGO

```
Registry registry = LocateRegistry.getRegistry(
    ipServidor, Main.PORTA);
IServidor servidor = (IServidor)
    registry.lookup(Main.NOME_SERVICO);
```

1 · Registro

Cliente encontra o serviço pelo nome `JogoDaVelha`.

2 · Pedido

Envia IP, porta TCP e protocolo `tcp` em JSON.

3 · Espera

Se não há par, o pedido fica pendente até match ou timeout.

4 · Retorno

IP/porta do rival e `inicia`.

O Registry é criado pelo próprio programa: LocateRegistry.createRegistry(Main.PORTA).

Fila de pedidos, sincronização e timeout

A lista de pendentes permite que duas chamadas RMI sejam completadas em momentos diferentes.

TRECHO DO CÓDIGO

```
private final Object lock = new Object();
private final List<Pedido> pendentes =
    new ArrayList<>();

synchronized (lock) {
    // procurar protocolo compatível
    // remover o pedido pareado
    // definir quem inicia
    lock.notifyAll();
}
```

- Valida o JSON: IP, porta e protocolo.
- Só pareia pedidos com o mesmo protocolo.
- O primeiro jogador aguardando recebe `inicia=false`; o que chega depois recebe `true`.
- Usa `wait()` / `notifyAll()` para liberar a chamada pendente.
- Timeout de 120 segundos remove o pedido sem par.

Por que sincronizar?

Duas requisições podem chegar quase simultaneamente. O lock protege a lista e evita pareamentos inconsistentes.

Por que TCP para as jogadas?

O código usa uma conexão direta entre os dois jogadores após o pareamento.

TRECHO DO CÓDIGO

```
serverSocket = new ServerSocket(0);
int minhaPorta = serverSocket.getLocalPort();

if (inicia) {
    socket = new Socket(ipAdversario,
                        portaAdversario);
} else {
    socket = serverSocket.accept();
}
```

Entrega confiável

TCP retransmite segmentos perdidos e oferece um fluxo de bytes ordenado.

JSON delimitado por linha

PrintWriter.println() envia uma mensagem; BufferedReader.readLine() lê até a quebra de linha.

Trade-off

Há conexão e estado por partida, mas não é preciso tratar perda/reordenação de datagramas como no UDP.

Porta 0 = o sistema operacional escolhe uma porta TCP livre; o jogador informa essa porta ao servidor de match.

JSON simples e protocolo de aplicação

O mesmo formato transporta estado suficiente para validar e reproduzir a jogada.

TRECHO DO CÓDIGO

```
{
  "Numero": 2,
  "Jogada": 4,
  "Tabuleiro": [1, 2, 0, 0, 1, 0, 0, 0, 2],
  "Resultado": "jog"
}
```

- `Numero`: sequência para conferir a ordem das jogadas.
- `Jogada`: posição escolhida no tabuleiro.
- `Tabuleiro`: estado de nove casas após a jogada.
- `Resultado`: `jog`, `vit`, `der` ou `emp`.

Validação no jogador

Confere o número esperado, se a posição está entre 0 e 8 e se a casa ainda está vazia.

Uma publicação, vários displays

Os displays não participam do jogo: apenas recebem e exibem o estado publicado.

TRECHO DO CÓDIGO

```
byte[] dados = mapper.writeValueAsBytes(mensagem);
DatagramPacket pacote = new DatagramPacket(
    dados, dados.length,
    grupoMulticast, Main.PORTA_MULTICAST);
multicastSocket.send(pacote);
```

TRECHO DO CÓDIGO

```
MulticastSocket socket =
    new MulticastSocket(Main.PORTA_MULTICAST);
socket.joinGroup(grupo);

socket.receive(pacote);
exibirJogada(mensagem);
```

Endereço de grupo

`225.1.1.16`

Porta

`8816`

Propriedade

1 emissor → N receptores

UDP é leve e não estabelece conexão; a entrega multicast não é garantida.

O que cada classe faz

Separação de papéis facilita explicar e testar cada parte do sistema.

Main

Define constantes (porta, nome do serviço, grupo multicast) e seleciona o modo de execução.

IServidor

Contrato remoto: `getOponent(String jsonIn)` retorna uma resposta JSON.

Servidor

Valida pedidos, mantém pendentes, combina jogadores e retorna IP/porta/inicia.

Jogador

Coordena o fluxo da partida: pareamento RMI, socket TCP, regras do jogo, JSON e envio multicast.

Visualizador

Entra no grupo multicast, recebe datagramas, interpreta JSON e redesenha o tabuleiro no terminal.

A classe Main escolhe `servidor`, `jogador` ou `visualizador` pelos argumentos da linha de comando.

Do comando à última jogada

O RMI é usado no início; depois o jogo continua diretamente entre os jogadores.

1 · Inicia

Servidor cria Registry e publica `JogoDaVelha`.

2 · Procura

Jogador abre porta TCP e chama `getOponent` por RMI.

3 · Pareia

Servidor combina dois pedidos e decide quem começa.

4 · Joga

Jogadores trocam JSON por TCP, alternando turnos.

5 · Publica

Cada jogada é enviada ao grupo multicast.

6 · Exibe

Displays atualizam o tabuleiro; fim em vitória/empate.

A conexão TCP termina quando a partida acaba; o visualizador pode continuar ouvindo o grupo.

Por que essa combinação funciona?

Cada mecanismo resolve um problema diferente; nenhum substitui os outros.

RMI · controle

- Chamada de método tipada
- Registry para localizar serviço
- Adequado para match, não para cada jogada

TCP · partida

- Confiável e ordenado
- Conexão dedicada entre o par
- Mensagens JSON terminadas em newline

Multicast UDP · observação

- Um envio pode chegar a vários displays
- Sem conexão individual por display
- Melhor esforço, sem garantia de entrega

Limitação a reconhecer

Multicast depende da configuração da rede; RMI também precisa anunciar um IP alcançável e permitir a porta do objeto remoto, além da porta do Registry.

Demonstração e conclusão

A proposta é uma integração de protocolos para uma aplicação distribuída pequena.

- Executar o servidor de match na VM .16.
- Abrir um visualizador e confirmar que está aguardando multicast.
- Iniciar dois jogadores em VMs diferentes.
- Fazer algumas jogadas e observar o tabuleiro no display.
- Mostrar o fim por vitória ou empate.

RMI

Descobre o oponente e define a ordem inicial.

TCP

Transporta as jogadas entre os dois participantes.

Multicast UDP

Distribui atualizações para quantos displays estiverem ouvindo.

Perguntas?